🚀 TP 3 : Base de Données et Déploiement en Production

🎯 Objectifs de l'Atelier

À la fin de cet atelier, vous aurez :

1. ✅ Migré le backend vers **PostgreSQL** avec SQLAlchemy ORM
2. ✅ Créé une base PostgreSQL gratuite sur **Supabase**
3. ✅ Mis en ligne le backend et le frontend sur **Render**
4. ✅ Mis en place le **déploiement continu** : chaque push testé par la CI est déployé automatiquement

---

📦 Architecture Cible

Avant (Local - Stockage en mémoire) :

```
Frontend (localhost:3000) ← → Backend (localhost:8000)
                                      ↓
                              Liste Python (RAM)
                              ❌ Données perdues au redémarrage
```

Après (Production avec PostgreSQL) :

```
Frontend (Render)                    Backend (Render)                    Database (Supabase)
taskflow-frontend-xxxx.onrender.com → taskflow-backend-xxxx.onrender.com → PostgreSQL
         HTTPS                                HTTPS + CORS                ✅ Données persistantes
```

✍️ Exercice 1 : Installer les Dépendances PostgreSQL

Objectif

Ajouter SQLAlchemy et le driver PostgreSQL au backend.

Instructions

Ajoutez les packages nécessaires :

```
cd backend
uv add sqlalchemy psycopg2-binary
```

💡 Ce que font ces packages

- `sqlalchemy` : ORM (Object-Relational Mapping) pour Python - permet de manipuler la base de données avec des objets Python
- `psycopg2-binary` : Driver PostgreSQL - permet à Python de se connecter à PostgreSQL

✍️ Exercice 2 : Configurer la Base de Données

Objectif

Créer le fichier de configuration pour la connexion à la base de données.

Instructions

Créez le fichier `backend/src/database.py` :

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# Lire l'URL de la base de données depuis les variables d'environnement
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./taskflow.db")

# Configuration du moteur SQLAlchemy
if DATABASE_URL.startswith("sqlite"):
    # SQLite (développement local)
    engine = create_engine(
        DATABASE_URL,
        connect_args={"check_same_thread": False}
    )
else:
    # PostgreSQL (production)
    engine = create_engine(
        DATABASE_URL,
        pool_size=5,
        max_overflow=10,
        pool_pre_ping=True
    )

# Factory de sessions
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Base pour les modèles ORM
Base = declarative_base()


def get_db():
    """Générateur qui fournit une session de base de données."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def init_db():
    """Initialise la base de données en créant toutes les tables."""
    from . import models  # Import des modèles pour créer les tables
    Base.metadata.create_all(bind=engine)
```

💡 Points importants

- `DATABASE_URL` : URL de connexion (SQLite en local, PostgreSQL en production)
- **Pool de connexions** : Réutilise les connexions pour améliorer les performances
- `pool_pre_ping` : Vérifie que la connexion est vivante avant de l'utiliser

---

✍️ Exercice 3 : Créer le Modèle de Données

Objectif

Définir le schéma de la table `tasks` avec SQLAlchemy ORM.

Instructions

Créez le fichier `backend/src/models.py` :

```
from enum import Enum
from sqlalchemy import Column, String, DateTime, Enum as SQLEnum
from sqlalchemy.sql import func

from .database import Base


class TaskStatus(str, Enum):
    """Statuts possibles d'une tâche."""
    TODO = "todo"
    IN_PROGRESS = "in_progress"
    DONE = "done"


class TaskPriority(str, Enum):
    """Priorités possibles d'une tâche."""
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"


class TaskModel(Base):
    """Modèle SQLAlchemy pour la table tasks."""
    __tablename__ = "tasks"

    id = Column(String, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    description = Column(String(1000), nullable=True)
    status = Column(SQLEnum(TaskStatus), default=TaskStatus.TODO)
    priority = Column(SQLEnum(TaskPriority), default=TaskPriority.MEDIUM)
    assignee = Column(String(100), nullable=True)
    due_date = Column(DateTime, nullable=True)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now())
```

💡 Avantages de l'ORM

- Pas besoin d'écrire du SQL directement
- Type-safety avec Python
- Migrations de schéma facilitées
- Timestamps automatiques (`created_at` , `updated_at`)

✍️ Exercice 4 : Migrer l'Application vers PostgreSQL

Objectif

Adapter `app.py` pour utiliser SQLAlchemy au lieu du stockage en mémoire.

Contexte

Actuellement, `app.py` stocke les tâches dans un dictionnaire Python (`tasks_db`). Au redémarrage du serveur, toutes les données sont perdues. Nous allons migrer vers SQLAlchemy pour persister les données dans PostgreSQL.

Instructions

Étape 1 : Mettre à jour les imports

Ouvrez `backend/src/app.py` et **ajoutez** les imports nécessaires (gardez les imports existants !) :

```
import uuid
from fastapi import Depends
from pydantic import ConfigDict
from sqlalchemy.orm import Session
from sqlalchemy import text

from .database import get_db, init_db
from .models import TaskModel, TaskStatus, TaskPriority
```

💡 **Pourquoi ces imports ?**

- `Depends` : Injection de dépendances FastAPI pour la session DB
- `Session` : Type de la session SQLAlchemy
- `text` : Pour exécuter du SQL brut (health check)
- `uuid` : Pour générer des identifiants uniques
- `ConfigDict` : Pour configurer le modèle Pydantic `Task` (étape 4)

Étape 2 : Nettoyer le code obsolète

Cherchez et supprimez ces éléments dans `app.py` :

```
# ❌ SUPPRIMER : Ces classes
class TaskStatus(str, Enum):
    TODO = "todo"
    ...

class TaskPriority(str, Enum):
    LOW = "low"
    ...

# ❌ SUPPRIMER : Le stockage en mémoire
tasks_db: Dict[int, Task] = {}
next_id = 1

# ❌ SUPPRIMER : Ces fonctions
def get_next_id() -> int:
    ...

def clear_tasks():
    ...

# ⚠️ À MODIFIER : Le health check (sera réécrit à l'étape 6)
@app.get("/health")
async def health_check():
    return {"status": "healthy", "tasks_count": len(tasks_db)}
```

> 💡 *Ces éléments sont maintenant dans `models.py` ou remplacés par SQLAlchemy.*

Étape 3 : Créer les tables au démarrage

La fonction `lifespan` (juste avant `app = FastAPI(...)`) s'exécute une fois au démarrage du serveur (avant `yield`) et une fois à l'arrêt (après `yield`). Modifiez-la pour créer les tables au démarrage :

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Runs once at startup (before yield) and once at shutdown (after yield)."""
    logger.info("🚀 TaskFlow backend starting up...")
    init_db()  # ← Nouveau : crée les tables si elles n'existent pas
    logger.info("✅ Database initialized")  # ← remplace "Using in-memory storage"
    yield
    logger.info("🛑 TaskFlow backend shutting down...")
```

Étape 3b : Ajouter le middleware CORS

Ajoutez ces imports en haut du fichier (avec les autres imports) :

```
from fastapi.middleware.cors import CORSMiddleware
import os
```

Puis ajoutez ce code **juste après** `app = FastAPI(...)` :

```
# Configuration CORS pour le frontend
cors_origins_str = os.getenv("CORS_ORIGINS", "http://localhost:5173,http://localhost:3000")
# strip() enlève les espaces, rstrip("/") enlève un éventuel "/" final
cors_origins = [origin.strip().rstrip("/") for origin in cors_origins_str.split(",")]

app.add_middleware(
    CORSMiddleware,
    allow_origins=cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

> ⚠️ ***Important*** *: Le middleware CORS est essentiel pour que le frontend puisse communiquer avec le backend en production. La variable* `CORS_ORIGINS` *sera configurée sur Render à l'exercice 7 (étape 3).*

Étape 4 : Modifier la classe Task existante

Cherchez la classe `Task` dans `app.py` et remplacez-la :

```
# ❌ AVANT (ne fonctionne plus avec SQLAlchemy)
class Task(TaskCreate):
    id: int  # ← int ne marche pas avec UUID
    created_at: datetime
    updated_at: datetime

# ✅ APRÈS (compatible SQLAlchemy)
class Task(BaseModel):
    """Model for task response."""
    id: str  # ← Changé en str pour UUID
    title: str
    description: Optional[str] = None
    status: TaskStatus
    priority: TaskPriority
    assignee: Optional[str] = None
    due_date: Optional[datetime] = None
    created_at: datetime
    updated_at: datetime
```

```
    model_config = ConfigDict(from_attributes=True)  # Permet la conversion depuis SQLAlchemy
```

> 💡 ***Pourquoi ces changements ?***
>
> - `id: str` *au lieu de* `int` → *les UUID sont des strings*
> - `ConfigDict(from_attributes=True)` → *permet à Pydantic de lire les objets SQLAlchemy*

Étape 5 : Modifier les endpoints

Pour chaque endpoint, ajoutez `db: Session = Depends(get_db)` comme paramètre.

Exemple simple avec health check :

```
# ⚠️ AVANT (ne fonctionne plus)
@app.get("/health")
async def health_check():
    return {"status": "healthy", "tasks_count": len(tasks_db)}

# ✅ APRÈS (avec SQLAlchemy)
@app.get("/health")
async def health_check(db: Session = Depends(get_db)):
    """Health check with database status."""
    try:
        db.execute(text("SELECT 1"))
        tasks_count = db.query(TaskModel).count()
        return {
            "status": "healthy",
            "database": "connected",
            "tasks_count": tasks_count
        }
    except Exception as e:
        return {"status": "unhealthy", "database": str(e)}
```

> ⚠️ ***Attention*** *:* `db: Session = Depends(get_db)` *doit être dans les* ***paramètres*** *de la fonction, pas dans le corps !*

Exemple complet avec GET /tasks :

```
@app.get("/tasks", response_model=List[Task])
async def get_tasks(
    status: Optional[TaskStatus] = None,
    priority: Optional[TaskPriority] = None,
    assignee: Optional[str] = None,
    db: Session = Depends(get_db)  # ← Toujours en dernier dans les paramètres
):
    """Get all tasks with optional filtering."""
    query = db.query(TaskModel)

    if status:
        query = query.filter(TaskModel.status == status)
    if priority:
        query = query.filter(TaskModel.priority == priority)
    if assignee:
        query = query.filter(TaskModel.assignee == assignee)

    return query.all()
```

> ⚠️ ***Ne confondez pas*** `Task` ***et*** `TaskModel` ***!***

> | *Modèle* | *Type* | *Utilisation* |
> |---|---|---|
> | `Task` | *Pydantic* | `response_model=Task` *(réponses API)* |
> | `TaskModel` | *SQLAlchemy* | `db.query(TaskModel)`, `TaskModel(...)` *(opérations DB)* |
>
> ***Règle simple** : Pour tout ce qui touche à la base de données → `TaskModel`*

Adaptez chaque endpoint selon ce tableau :

Endpoint	Logique SQLAlchemy
GET /tasks	`db.query(TaskModel).all()`
GET /tasks/{id}	`db.query(TaskModel).filter(TaskModel.id == task_id).first()`
POST /tasks	`TaskModel(id=str(uuid.uuid4()), ...)` → `db.add()` → `db.commit()` → `db.refresh()`
PUT /tasks/{id}	`setattr(task, field, value)` → `db.commit()` → `db.refresh(task)`
DELETE /tasks/{id}	`db.delete(task)` → `db.commit()`

> 💡 ***Pattern SQLAlchemy :***
>
> 1. `db.add(obj)` *- Ajoute à la session*
> 2. `db.commit()` *- Sauvegarde en base*
> 3. `db.refresh(obj)` *- Recharge les valeurs générées (timestamps, etc.)*
>
> ***Indice POST /tasks** : Créez un `TaskModel` (pas `Task`) avec `id=str(uuid.uuid4())` au lieu de `get_next_id()`*

💡 Points clés à retenir

Concept	Explication
`Depends(get_db)`	FastAPI injecte automatiquement une session DB
`db.commit()`	Obligatoire pour sauvegarder les changements
`db.refresh()`	Recharge l'objet avec les valeurs de la DB (timestamps)
`ConfigDict(from_attributes=True)`	Permet à Pydantic de lire les attributs SQLAlchemy

✅ Checkpoint

Testez localement :

```
cd backend
uv run uvicorn src.app:app --reload

# Dans un autre terminal
curl http://localhost:8000/health
curl http://localhost:8000/tasks
```

Vous devriez voir un fichier `taskflow.db` créé dans `backend/`

✍️ Exercice 5 : Adapter les Tests

Objectif

Modifier les tests pour utiliser une base de données SQLite temporaire.

Contexte

Les tests utilisent actuellement `clear_tasks()` qui n'existe plus. Nous devons créer une base de données de test isolée et nettoyer les données entre chaque test.

Instructions

Remplacez le contenu de `backend/tests/conftest.py` par :

```
import pytest
import tempfile
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from src.app import app
from src.database import Base, get_db
from src.models import TaskModel

TEST_DB_FILE = tempfile.mktemp(suffix=".db")
TEST_DATABASE_URL = f"sqlite:///{TEST_DB_FILE}"

test_engine = create_engine(
    TEST_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)

TestSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=test_engine)


@pytest.fixture(scope="session")
def setup_test_database():
    """Crée les tables une seule fois pour tous les tests."""
    Base.metadata.create_all(bind=test_engine)
    yield
    Base.metadata.drop_all(bind=test_engine)


@pytest.fixture(autouse=True)
def clear_test_data(setup_test_database):
    """Nettoie les données entre chaque test."""
    db = TestSessionLocal()
    db.query(TaskModel).delete()
    db.commit()
    db.close()


@pytest.fixture
def client(setup_test_database):
    """Client de test avec base de données isolée."""
    def override_get_db():
        db = TestSessionLocal()
        try:
```

```
            yield db
        finally:
            db.close()

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()
```

> 💡 *`dependency_overrides` permet de remplacer `get_db` par une version qui utilise la base de test au lieu de la vraie base.*

✅ Checkpoint

```
cd backend
uv run pytest -v
```

Tous les tests qui passaient avec le stockage en mémoire doivent toujours passer : la base de données a changé, pas le comportement de l'API.

✍️ Exercice 6 : Créer la Base de Données sur Supabase

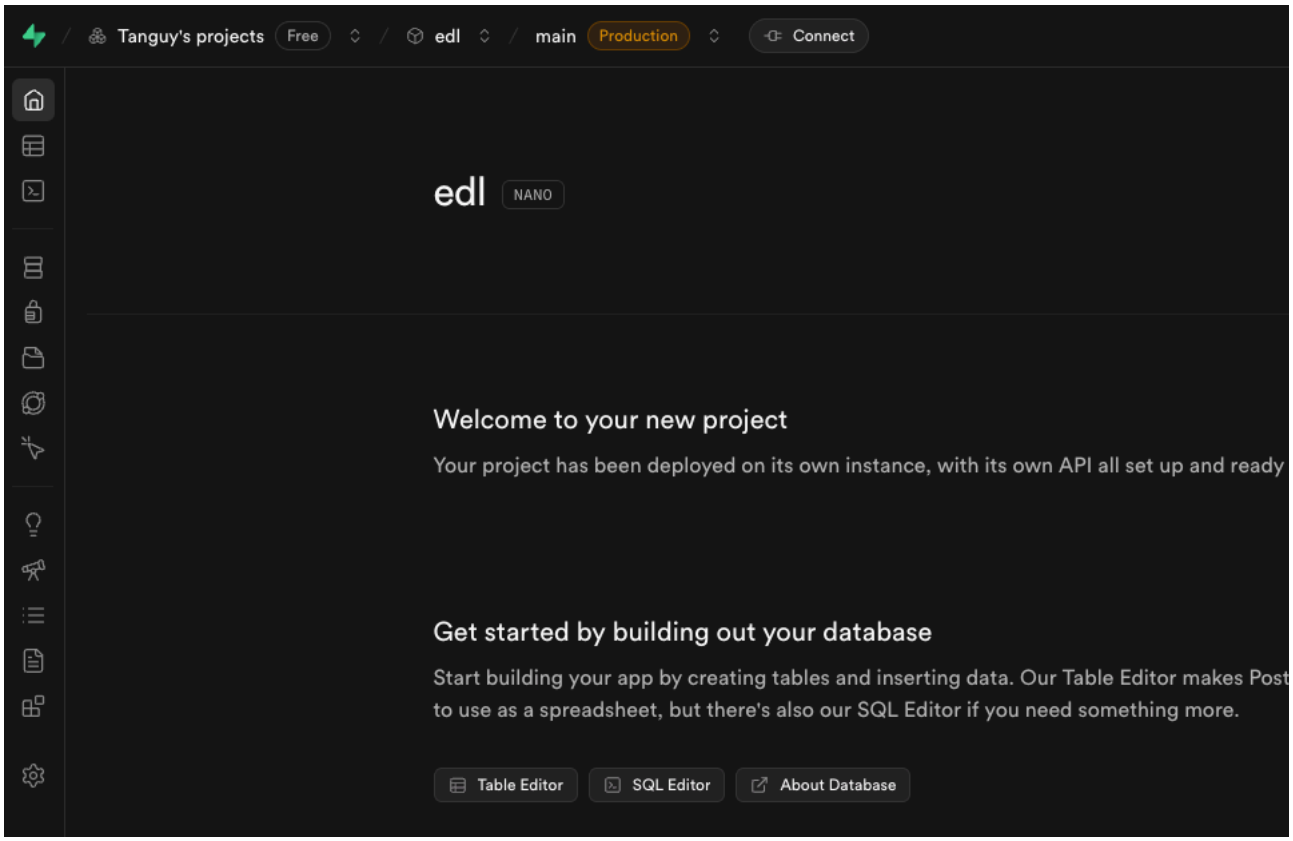
Objectif

Créer une base PostgreSQL gratuite sur Supabase, puis vérifier **en local** que votre backend arrive à s'y connecter. On ne déploie rien tant que ça ne marche pas en local.

Étape 1 : Créer un compte et un projet

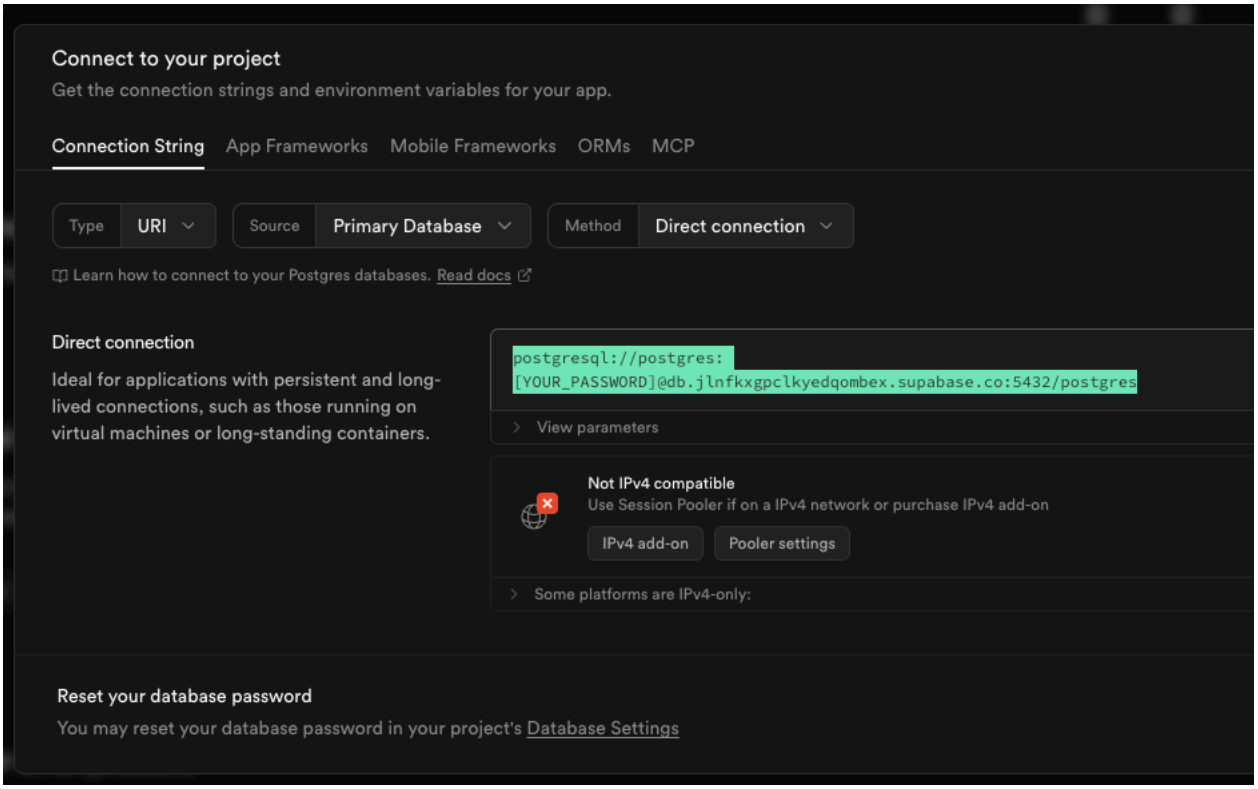
1. Allez sur https://supabase.com → **"Start your project"** → inscrivez-vous avec **GitHub**
2. Cliquez **"New project"** et configurez :
   - **Name** : `taskflow`
   - **Database Password** : un mot de passe composé **uniquement de lettres et de chiffres** (⚠️ **notez-le !**)
   - **Region** : `West EU (Ireland)`
3. Cliquez **"Create new project"** et attendez 1-2 minutes

> ⚠️ ***Pourquoi seulement des lettres et des chiffres ?*** *Le mot de passe va être placé dans une URL. Les caractères comme `@ # / : ? %` ont un sens spécial dans une URL et cassent la connexion.*



Étape 2 : Récupérer la DATABASE_URL

1. Cliquez sur le bouton **"Connect"** en haut de la page



2. Dans l'onglet **Connection String** :
   - **Type** : `URI`
   - **Method** : `Session pooler` (⚠️ pas "Direct connection")

3. Copiez l'URL et remplacez `[YOUR-PASSWORD]` par votre mot de passe (sans les crochets)

L'URL ressemble à :

```
postgresql://postgres.abcdefghijkl:MonMotDePasse123@aws-0-eu-west-1.pooler.supabase.com:5432/postgres
```

> 💡 ***Pourquoi "Session pooler" ?*** *Render ne parle qu'en IPv4, alors que la connexion directe de Supabase demande l'IPv6. Le pooler fait le pont.*

Étape 3 : Tester la connexion en local

Lancez votre backend en lui donnant cette URL (remplacez par la vôtre, **entre guillemets**) :

macOS / Linux :

```
cd backend
DATABASE_URL="postgresql://postgres.xxx:MotDePasse@aws-0-eu-west-1.pooler.supabase.com:5432/postgres" u
```

Windows (PowerShell) :

```
cd backend
$env:DATABASE_URL="postgresql://postgres.xxx:MotDePasse@aws-0-eu-west-1.pooler.supabase.com:5432/postgr
uv run uvicorn src.app:app --reload
```

Ouvrez http://localhost:8000/health. Vous devez voir :

```
{"status": "healthy", "database": "connected", "tasks_count": 0}
```

Créez une tâche via http://localhost:8000/docs, puis ouvrez **Table Editor** dans Supabase : la table `tasks` contient votre tâche. 🎉

❌ **Ça ne marche pas ?** Voir le tableau Problèmes fréquents en fin de TP. Ne passez pas à la suite avant d'avoir `"database": "connected"`.

⚠️ **Gardez cette URL**, vous en aurez besoin à l'exercice 7. Ne la commitez jamais dans Git : elle contient votre mot de passe.

✍️ Exercice 7 : Déployer sur Render

Objectif

Mettre le backend et le frontend en ligne. L'**ordre** est important, car chaque service a besoin de l'URL de l'autre :

```
1. Backend   → on obtient l'URL du backend
2. Frontend  → on lui donne l'URL du backend (VITE_API_URL)
               → on obtient l'URL du frontend
3. Backend   → on lui donne l'URL du frontend (CORS_ORIGINS)
```

Recopiez ce tableau dans un fichier texte et complétez-le au fur et à mesure :

	Valeur
URL Supabase (`DATABASE_URL`)	`postgresql://...` (exercice 6)
URL du backend	`https://taskflow-backend-....onrender.com`

	Valeur
URL du frontend	`https://taskflow-frontend-....onrender.com`

> 💡 *Render ajoute souvent quelques caractères au nom (ex : `taskflow-backend-x7k2`) car le nom est déjà pris par quelqu'un d'autre. **Recopiez toujours l'URL exacte affichée par Render**, ne la devinez pas.*

Prérequis

1. Vos tests passent en local (`uv run pytest`) et votre CI du TP 2 est verte sur `main`
2. Poussez votre travail (y compris `pyproject.toml` et `uv.lock`, modifiés par `uv add`) :

```
git add .
git commit -m "feat: migrate to PostgreSQL with SQLAlchemy"
git push origin main
```

3. Créez un compte sur https://render.com avec **GitHub** et autorisez l'accès à votre repository

Étape 1 : Déployer le Backend

1. Sur le Dashboard Render : **"New +"** → **"Web Service"** → choisissez votre repository
2. Configurez :

Paramètre	Valeur
Name	`taskflow-backend`
Region	`Frankfurt (EU Central)`
Branch	`main`
Root Directory	`backend`
Runtime	`Python 3`
Build Command	`pip install uv && uv sync`
Start Command	`uv run uvicorn src.app:app --host 0.0.0.0 --port $PORT`
Instance Type	`Free`

> 💡 *Laissez `$PORT` tel quel : Render le remplace lui-même.*

3. Dans **Environment Variables**, ajoutez **une seule** variable pour l'instant :

Name	Value
`DATABASE_URL`	Votre URL Supabase (exercice 6)

4. Cliquez **"Create Web Service"** et suivez les logs. Attendez le statut **Live** (3-5 min).
5. Copiez l'**URL du backend** affichée en haut de la page (ex : `https://taskflow-backend-x7k2.onrender.com`) dans votre tableau.
6. Vérifiez : ouvrez `<URL du backend>/health` dans le navigateur → `"database": "connected"`.

Étape 2 : Déployer le Frontend

1. **"New +"** → **"Static Site"** → même repository
2. Configurez :

Paramètre	Valeur
Name	`taskflow-frontend`
Branch	`main`
Root Directory	`frontend`
Build Command	`npm ci && npm run build`
Publish Directory	`dist`

3. Dans **Environment Variables**, ajoutez :

Name	Value
`VITE_API_URL`	**Votre** URL du backend (étape 1)

4. Cliquez **"Create Static Site"**, attendez **Live**, puis copiez l'**URL du frontend** dans votre tableau.

> ⚠️ `VITE_API_URL` ***est lue au moment du build****, puis écrite "en dur" dans les fichiers JavaScript. Si vous la modifiez après coup, il faut* ***reconstruire*** *le frontend :* ***Manual Deploy*** *→* ***Deploy latest commit****.*

Étape 3 : Autoriser le Frontend (CORS)

Par sécurité, le navigateur refuse qu'une page d'un site (le frontend) appelle un autre site (le backend), sauf si le backend l'autorise. C'est le rôle de `CORS_ORIGINS`.

1. Dashboard → **taskflow-backend** → **Environment** → ajoutez :

Name	Value
`CORS_ORIGINS`	**Votre** URL du frontend (étape 2)

2. **"Save Changes"** → le backend redémarre automatiquement.

Étape 4 : Ouvrir l'Application

Ouvrez votre **URL du frontend**. Vous devez voir TaskFlow, connecté à votre backend en ligne. 🎉

> ⏳ ***Le premier chargement peut prendre ~1 minute.*** *Sur l'offre gratuite, Render met le backend en veille après 15 minutes sans visite. S'il affiche "Erreur de Connexion", attendez un peu et rechargez la page.*

✍️ Exercice 8 : Vérifier le Déploiement

Objectif

Vérifier que tout fonctionne en production et que les données sont bien persistantes.

Instructions

1. **Testez l'API** (remplacez `<URL_BACKEND>` par votre URL du backend) :

```
curl <URL_BACKEND>/health
```

Résultat attendu :

```
{"status": "healthy", "database": "connected", "tasks_count": 0}
```

2. **Testez le frontend** : créez, modifiez et supprimez quelques tâches.
3. **Vérifiez la persistance** :
   - Dans Supabase → **Table Editor** → `tasks` : vos tâches sont là
   - Sur Render → **taskflow-backend** → **Manual Deploy** → **Restart service**
   - Rechargez le frontend : vos tâches sont **toujours là** (contrairement au TP 1-2 où elles disparaissaient au redémarrage)

✍️ Exercice 9 : Déploiement Continu (CD)

Objectif

Faire en sorte que chaque push sur `main` soit **testé par GitHub Actions (TP 2), puis déployé par Render** — mais seulement si les tests passent.

Instructions

1. Sur Render → **taskflow-backend** → **Settings** → **Auto-Deploy** : choisissez **"After CI Checks Pass"**. Faites de même pour **taskflow-frontend**.
2. Modifiez le message de bienvenue dans `backend/src/app.py` :

```
"message": "Welcome to TaskFlow API - déployé automatiquement !",
```

3. Adaptez le test `test_root_endpoint` si nécessaire, vérifiez en local (`uv run pytest`), puis poussez :

```
git add .
git commit -m "feat: update welcome message"
git push origin main
```

4. Observez :
   - GitHub → **Actions** : vos workflows tournent
   - Render → **Events** : le déploiement démarre **après** que les checks soient verts
5. Vérifiez avec `curl <URL_BACKEND>/` : le nouveau message apparaît.

> 💡 ***Testez le cas contraire :*** *poussez un test qui échoue. La CI devient rouge et Render* ***ne déploie pas****. La production reste protégée.*

🎁 Pour Aller Plus Loin : Infrastructure as Code avec `render.yaml`

Au lieu de cliquer dans le Dashboard, on peut décrire les services dans un fichier versionné dans Git. Voici l'équivalent de l'exercice 7 :

```yaml
services:
  - type: web
    name: taskflow-backend
    runtime: python
    region: frankfurt
    plan: free
    rootDir: backend
    buildCommand: pip install uv && uv sync
    startCommand: uv run uvicorn src.app:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health
    envVars:
      - key: DATABASE_URL
        sync: false  # Render vous demandera la valeur
      - key: CORS_ORIGINS
        sync: false

  - type: web
    name: taskflow-frontend
    runtime: static
    rootDir: frontend
    buildCommand: npm ci && npm run build
    staticPublishPath: dist
    envVars:
      - key: VITE_API_URL
        sync: false
```

Pour l'utiliser : placez-le à la racine du repository sous le nom `render.yaml`, poussez-le, puis **"New +"** → **"Blueprint"** sur Render.

> ⚠️ *Un Blueprint crée de **nouveaux** services. Si vous avez déjà fait l'exercice 7, supprimez d'abord les anciens services (ou changez les `name`), sinon vous aurez des doublons.*

Aspect	Dashboard (exercice 7)	`render.yaml`
Comprendre les étapes	✅ Chaque paramètre est visible	⚠️ Tout est caché dans le fichier
Reproductible	❌ À refaire à la main	✅ Versionné dans Git
En entreprise	Pour tester	✅ Bonne pratique

🦠 Problèmes Fréquents

Où chercher les erreurs ? Backend : Render → **taskflow-backend** → **Logs**. Frontend : ouvrez la console du navigateur (**F12** → onglet **Console**).

Symptôme	Cause probable	Solution
`password authentication failed` (logs backend ou en local)	Mauvais mot de passe dans `DATABASE_URL`, ou crochets `[ ]` laissés autour	Recopiez l'URL depuis Supabase et remplacez `[YOUR-PASSWORD]` par le mot de passe seul
`Network is unreachable` ou timeout vers `db.xxx.supabase.co`	URL "Direct connection" (IPv6)	Reprenez l'URL en mode **Session pooler** (host `pooler.supabase.com`)

Symptôme	Cause probable	Solution
Le mot de passe contient `@`, `#`, `/` ...	Caractères spéciaux dans l'URL	Supabase → **Project Settings** → **Database** → **Reset database password** avec lettres et chiffres uniquement
`ModuleNotFoundError: No module named 'sqlalchemy'` sur Render	`pyproject.toml` / `uv.lock` pas poussés après `uv add`	`git add backend/pyproject.toml backend/uv.lock`, commit, push
Build backend : fichier introuvable / `src.app` introuvable	**Root Directory** vide	Settings → Root Directory = `backend`
Frontend : "Erreur de Connexion" au premier chargement	Backend en veille (offre gratuite)	Attendez ~1 minute et rechargez
Console : requêtes vers `/api/tasks` en **404**	`VITE_API_URL` absente au moment du build	Ajoutez-la, puis **Manual Deploy** → **Deploy latest commit** du frontend
Console : `blocked by CORS policy`	`CORS_ORIGINS` absente ou différente de l'URL du frontend (faute de frappe, `http` au lieu de `https`)	Copiez l'URL exacte du frontend dans `CORS_ORIGINS`
Le frontend appelle le backend de quelqu'un d'autre	URL d'un exemple ou d'un camarade copiée dans `VITE_API_URL`	Mettez **votre** URL du backend et reconstruisez le frontend
Exercice 9 : Render ne déploie jamais	Un workflow GitHub Actions est rouge	Corrigez les tests : Render attend que **tous** les checks soient verts

---

📋 Récapitulatif

Félicitations ! Vous avez maintenant :

✅ **Exercice 1** : Installé SQLAlchemy et psycopg2 ✅ **Exercice 2** : Configuré la connexion à la base de données ✅ **Exercice 3** : Créé le modèle ORM TaskModel ✅ **Exercice 4** : Migré app.py vers PostgreSQL ✅ **Exercice 5** : Adapté les tests avec une DB temporaire ✅ **Exercice 6** : Créé la base Supabase et testé la connexion en local ✅ **Exercice 7** : Déployé backend et frontend sur Render ✅ **Exercice 8** : Vérifié le déploiement et la persistance des données ✅ **Exercice 9** : Mis en place le déploiement continu (CI verte → déploiement)

Temps total estimé : 3h

📚 Ce que Vous Avez Appris

✅ **SQLAlchemy ORM** - Modèles Python ↔ Tables SQL ✅ **PostgreSQL** - Base de données relationnelle professionnelle ✅ **Déploiement Cloud** - Backend et Frontend sur Render ✅ **Base de données managée** - Supabase pour PostgreSQL ✅ **Variables d'environnement** - Configuration production vs développement ✅ **CORS** - Communication cross-origin frontend/backend ✅ **Déploiement continu** - La CI protège la production